A Real-Time Multithreaded ER (Emergency Room) Triage Simulator

Team Members

Ferdi Fadillah ff2364
Sruthi Raghavan sr7892

Problem Statement

Emergency departments face a constant challenge: patients arrive unpredictably, each presenting with varying degrees of medical urgency, while the number of available doctors and treatment rooms remains limited. Without a structured prioritization system, critical patients risk delayed care, so that it can cause a potentially life-threatening outcome.

The ER triage process exists to address this by classifying patients based on urgency and directing them to care in the appropriate order. However, managing this process manually, which is tracking queue status, room availability, and patient state simultaneously, is error-prone and difficult to optimize in real time.

This project proposes a real-time, multithreaded ER (Emergency Room) Triage Simulator built with Java and Spring Boot. The system models concurrent patient intake and treatment workflows, allowing hospital staff to monitor live queue status, room occupancy, and patient triage levels through a dynamic interface, supporting better decision-making and more efficient patient flow management.

Literature Review

The Five ESI (Emergency Severity Index) Levels

The ESI algorithm categorizes patients by acuity while also considering the resources needed for stable, less urgent patients. Nursing CE Central The five levels are described below:

Level	Name	Description	Example Conditions
ESI 1	Immediate	Requires life-saving intervention right now. The patient is unstable.	Cardiac arrest, respiratory failure, unresponsive trauma
ESI 2	Emergent	High-risk situation;	Chest pain, stroke

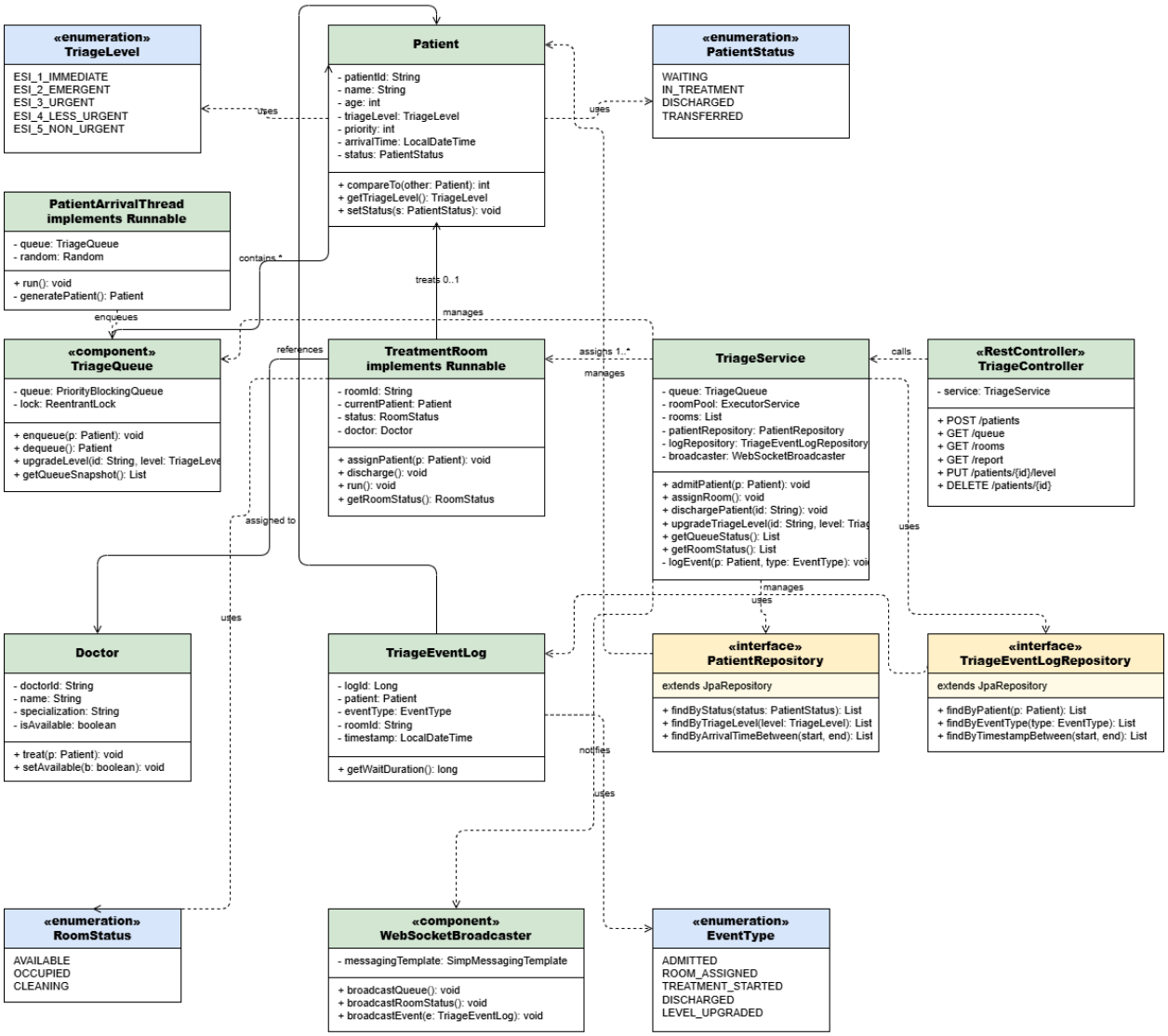
		patient in severe distress or confusion. Cannot wait.	symptoms, altered mental status, severe allergic reaction
ESI 3	Urgent	Stable but requires 2 or more resources (labs, imaging, IV, etc.).	Abdominal pain, fracture, moderate asthma attack, high fever
ESI 4	Less Urgent	Stable; requires only 1 resource.	Minor laceration needing stitches, urinary tract infection, sprained ankle
ESI 5	Non-Urgent	Stable; requires no resources. Simple assessment or prescription only.	Cold symptoms, minor rash, medication refill request

Technology Stack

Layer	Technology	Purpose
Backend Framework	Spring Boot	Core application framework, REST API, dependency injection
Language	Java	Primary programming language
Multithreading	Java ExecutorService, PriorityBlockingQueue, ReentrantLock	Concurrent treatment room simulation and thread-safe queue management
Real-Time Networking	Spring WebSocket + STOMP	Push live queue and room status updates to connected clients
REST API	Spring MVC (@RestController)	Patient intake, discharge, and triage update endpoints
ORM	Spring Data JPA + Hibernate	Entity mapping and database query abstraction
Security	Spring Security + JWT	Role-based access (Admin, Nurse, Doctor)
GUI / Frontend	Java Swing	Live dashboard showing queue, room status, and wait times

Build Tool	Maven	Dependency management and project build
Testing	JUnit 5 + Mockito	Unit and concurrency stress testing
API Testing	Postman	Manual REST endpoint testing
Version Control	Git + GitHub	Source control and team collaboration

Class Diagram



Advanced Concept

The project requires you to incorporate at least 3 advanced Java concepts from the course lectures or your own reading. These are things that go beyond basic Java syntax, data types, and OOP fundamentals. The document points to these areas as examples of what counts as "advanced":

1. GUI (Graphical User Interface)

Building a visual, interactive interface for your application — not just a command-line program. In Java this typically means Java Swing (JFrame, JTable, JPanel, etc.) or JavaFX. The Yahtzee and Snake Game examples are GUI-heavy projects.

2. Multithreading

Using Java threads to do multiple things concurrently. Examples from the doc include a multi-threaded server that handles multiple client connections simultaneously, or parallelizing an analytic process. Key classes: Thread, Runnable, ExecutorService, synchronized blocks, concurrent data structures.

3. Networking

Having your application communicate over a network — a client connecting to a server over a socket, or using HTTP/REST. The three-tier architecture example in the doc (UI → network → server → DB) specifically calls out network communication as one of its features.

4. Databases

Storing and retrieving data persistently. The doc references a DB layer where results can be stored and queried — your ER Triage project satisfies this with JPA/Spring Data.

5. REST API / Spring Framework

The doc explicitly mentions these as additional areas beyond the core lectures that allow you to build "other powerful applications." Using Spring Boot with REST controllers counts as an advanced concept on its own.

To Do List

Phase 1: Data Layer (Spring Data JPA)

- Add Spring Boot, Spring Data JPA, and H2/PostgreSQL dependencies in pom.xml
- Create Patient.java entity — fields: patientId, name, age, triageLevel (TriageLevel), priority, arrivalTime, status (PatientStatus)
- Create TriageEventLog.java entity — fields: logId, patient, eventType (EventType), roomId, timestamp; method: getWaitDuration()
- Create TriageLevel enum: ESI_1_IMMEDIATE, ESI_2_EMERGENT, ESI_3_URGENT, ESI_4_LESS_URGENT, ESI_5_NON_URGENT
- Create PatientStatus enum: WAITING, IN_TREATMENT, DISCHARGED, TRANSFERRED
- Create EventType enum: ADMITTED, ROOM_ASSIGNED, TREATMENT_STARTED, DISCHARGED, LEVEL_UPGRADED
- Create RoomStatus enum: AVAILABLE, OCCUPIED, CLEANING
- Create PatientRepository interface — extends JpaRepository<Patient, String>; methods: findByStatus(), findByTriageLevel(), findByArrivalTimeBetween()
- Create TriageEventLogRepository interface — extends JpaRepository<TriageEventLog, Long>; methods: findByPatient(), findByEventType(), findByTimestampBetween()
- Unit test all repository methods independently (H2 in-memory DB)

Phase 2: Core Multithreading

- Implement Comparable<Patient> on Patient so PriorityBlockingQueue sorts by ESI priority level
- Build TriageQueue component — wraps PriorityBlockingQueue<Patient> + ReentrantLock; methods: enqueue(), dequeue(), upgradeLevel(), getQueueSnapshot()
- Create Doctor.java model — fields: doctorId, name, specialization, isAvailable; methods: treat(Patient), setAvailable(boolean)
- Build PatientArrivalThread (implements Runnable) — randomly generates Patient instances and calls TriageQueue.enqueue(); method: generatePatient()
- Build TreatmentRoom (implements Runnable) — fields: roomId, currentPatient, status (RoomStatus), doctor (Doctor); methods: assignPatient(), discharge(), run(), getRoomStatus()
- Spin up 3–5 TreatmentRoom threads via ExecutorService in TriageService
- Test concurrent queue access; confirm no race conditions under load
- Log TREATMENT_STARTED and DISCHARGED events to DB from room threads via TriageEventLogRepository

Phase 3: Real-Time Push (Spring WebSocket / STOMP)

- Add spring-boot-starter-websocket dependency in pom.xml
- Configure Spring WebSocket + STOMP message broker

- Build WebSocketBroadcaster component — wraps SimpMessagingTemplate; methods: broadcastQueue(), broadcastRoomStatus(), broadcastEvent(TriageEventLog)
- Call broadcastQueue() and broadcastRoomStatus() from TriageService whenever state changes
- Build a JavaScript/frontend monitor client (or test with wscat) that subscribes to STOMP topics and receives live updates
- Test 3+ simultaneous client subscriptions
- Handle client disconnect gracefully without server error

Phase 4: GUI (Java Swing)

- Build main JFrame dashboard window
- Live queue JTable — displays waiting patients sorted by ESI priority
- Room status panel — JLabel per room showing patient name or "Available"
- Color-code rows by ESI: Red (ESI 1), Orange (ESI 2), Yellow (ESI 3), Green (ESI 4/5)
- Patient wait time counter updating live per row
- Manual patient entry form — name + ESI level input + Add button (calls TriageService.admitPatient())
- Wire javax.swing.Timer to refresh UI every 500 ms by polling TriageService.getQueueStatus() and getRoomStatus()

Note: The Swing GUI is an optional monitoring layer. No GUI classes appear in the backend class diagram; the GUI calls into TriageService or polls the REST API.

Phase 5: Spring Boot REST API

- Set up Spring Boot application structure (controller, service, repositories already in diagram)
- Build TriageService — fields: queue (TriageQueue), roomPool (ExecutorService), rooms, patientRepository, logRepository, broadcaster; methods: admitPatient(), assignRoom(), dischargePatient(), upgradeTriageLevel(), getQueueStatus(), getRoomStatus(), private logEvent()
- Build TriageController (@RestController) — delegates to TriageService; endpoints:
    - POST /patients — admit a new patient
    - GET /queue — return current patient queue as JSON
    - GET /rooms — return current room statuses
    - GET /report — return wait time / event report from DB
    - PUT /patients/{id}/level — upgrade triage level
    - DELETE /patients/{id} — discharge/remove patient
- Test all REST endpoints with Postman or curl

Phase 6: Full Integration & Testing

- Connect all layers: Spring Boot + TriageQueue threads + JPA repositories + WebSocket broadcaster + Swing GUI (if included)
- Run full end-to-end simulation with all components live

- Fix any race conditions or deadlocks found during integration
- Stress test: simulate 50+ rapid patient arrivals simultaneously via PatientArrivalThread
- Verify all EventType values are correctly persisted to DB under concurrent load
- Test edge cases: empty queue, all rooms OCCUPIED, server restart recovery, patient TRANSFERRED status
- Add statistics/report screen in GUI pulling data from GET /report
- Write JavaDoc for all major classes and methods (Patient, TriageQueue, TreatmentRoom, TriageService, TriageController, WebSocketBroadcaster, all repositories)

Testing Strategy

Unit testing of individual components; the Patient comparator, DatabaseManager, and thread-safe queue operations; will be conducted using JUnit 5. Integration testing will verify that concurrent threads correctly interact with the shared priority queue and database under simultaneous load. Stress testing will simulate peak ER conditions with 50+ simultaneous patients to validate thread safety, queue ordering, and system stability. REST endpoints will be tested using Postman. Network testing will confirm that multiple clients can connect, receive live updates, and disconnect without destabilizing the server.

References

[1] Nursing CE Central, "Emergency Severity Index," *Nursing CE Central*, 2025. [Online]. Available: https://nursingcecentral.com/lessons/emergency-severity-index/. [Accessed: Apr. 4, 2026].

Provides an overview of the Emergency Severity Index (ESI), a five-level triage system used in emergency departments to classify patients based on urgency and required resources. It explains each ESI level with definitions and clinical examples, forming the basis for prioritizing patient care. This directly supports the simulator's design by defining how patients are categorized and ordered within the priority queue.

[2] A. Avinadav, T. Chernonog, and U. Perlman, "Queueing Problems in Emergency Departments: A Review of Practical Approaches and Research Methodologies," *Operations Research Forum*, vol. 2, no. 1, 2021.

Reviews practical approaches and research methodologies for analyzing queueing problems in emergency departments. The paper discusses how mathematical and operational models can optimize patient flow, reduce waiting times, and improve resource allocation in healthcare settings. This informs the simulator's approach to managing concurrent patient queues and optimizing treatment room utilization.

[3] X. Hu, S. Barnes, and B. Golden, "Applying queueing theory to the study of emergency department operations: a survey and a discussion of comparable simulation studies," *International Transactions in Operational Research*, vol. 25, no. 1, pp. 7–49, 2018.

Surveys the application of queueing theory and simulation in emergency department operations. It compares different modeling techniques and highlights how simulations can be used to evaluate system

performance and improve efficiency in patient handling. This supports the use of simulation in the project to test and analyze system performance under different patient loads.

[4] "Queueing theory and simulation for reducing patient waiting time in emergency departments," *ResearchGate*, 2026. [Online]. Available: https://www.researchgate.net/. [Accessed: Apr. 4, 2026].

Focuses on the use of queueing theory and simulation techniques to reduce patient waiting times in emergency departments. It emphasizes practical implementations and supports the use of computational models to enhance real-time decision-making and system performance. This reinforces the project's goal of building a real-time simulator to improve efficiency and reduce delays in emergency care**.**